

Dự đoán khả năng khách hàng rời bỏ ngân hàng bằng các thuật toán Machine Learning (Bank Customer Churn Prediction Using Machine Learning Algorithms)

Nguyễn Thị Quỳnh Như¹, Trần Quốc Bảo², Nguyễn Ngọc Hoài An³, Nguyễn Thành Long⁴

Nhóm 2 - IS403.Q11, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM
{23521128, 22520122, 22520022, 23520885}@gm.uit.edu.vn

Tóm tắt nội dung. Dự báo khách hàng rời bỏ (Customer Churn Prediction) là bài toán then chốt trong quản trị quan hệ khách hàng tại các ngân hàng hiện đại. Đồ án này xây dựng quy trình khai phá dữ liệu toàn diện để giải quyết vấn đề trên thông qua việc thực nghiệm và so sánh sáu thuật toán học máy: Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost và LightGBM. Kết quả nghiên cứu chỉ ra rằng các mô hình cơ bản (KNN, Decision Tree) mặc dù đạt độ chính xác tổng thể (Accuracy) cao ($\sim 84.7\%$) nhưng gặp phải "nghịch lý độ chính xác" với khả năng phát hiện khách hàng rời bỏ (Recall) thấp ($< 46\%$). Ngược lại, các kỹ thuật Ensemble và Boosting đã giải quyết hiệu quả vấn đề mất cân bằng dữ liệu, trong đó XGBoost được xác định là mô hình tối ưu nhất với chỉ số Recall đạt 72.7% và diện tích dưới đường cong ROC (AUC) đạt 0.865 . Bên cạnh đó, phân tích tầm quan trọng của đặc trưng (Feature Importance) cho thấy Độ tuổi (Age), Mức lương ước tính (EstimatedSalary) và Số dư (Balance) là các yếu tố dự báo quan trọng nhất. Từ những phát hiện định lượng này, nhóm nghiên cứu đề xuất các giải pháp giữ chân khách hàng mục tiêu dựa trên phân khúc độ tuổi và hành vi sử dụng sản phẩm.

Từ khóa: Churn Prediction, Machine Learning, Accuracy Paradox, XGBoost, Imbalanced Data.

1 Giới thiệu

1.1 Lý do chọn đề tài

Trong bối cảnh cạnh tranh gay gắt của ngành tài chính, việc giữ chân khách hàng đóng vai trò then chốt do chi phí thấp hơn và giá trị vòng đời cao hơn đáng kể so với việc thu hút mới [13]. Tận dụng nguồn dữ liệu khổng lồ từ hệ thống ngân hàng số, việc ứng dụng các thuật toán Học máy (Machine Learning) cho phép nhận diện sớm rủi ro rời bỏ (Customer Churn) với độ chính xác vượt trội so với thống kê truyền thống [3,9]. Đề tài này tập trung giải quyết bài toán trên nhằm cung cấp công cụ hỗ trợ ra quyết định kinh doanh dựa trên dữ liệu.

1.2 Mục tiêu nghiên cứu

Mục tiêu chính của nghiên cứu là xây dựng và đánh giá các mô hình học máy nhằm dự đoán khả năng rời bỏ của khách hàng ngân hàng dựa trên dữ liệu nhân khẩu học, tài chính và hành vi sử dụng dịch vụ. Cụ thể, nghiên cứu hướng đến các mục tiêu sau:

- Xử lý dữ liệu: Chuẩn hóa bộ dữ liệu về nhân khẩu, tài chính và hành vi để phục vụ bài toán phân loại.
- Đánh giá mô hình: Thực nghiệm và so sánh hiệu suất các thuật toán phân lớp, tập trung vào các chỉ số đo lường phù hợp với dữ liệu mất cân bằng (Recall, ROC-AUC).
- Đề xuất giải pháp: Xác định các yếu tố ảnh hưởng then chốt (Key Drivers) làm cơ sở đưa ra các hàm ý quản trị giúp ngân hàng giữ chân khách hàng hiệu quả.

1.3 Phạm vi nghiên cứu

Nghiên cứu tập trung giải quyết bài toán phân loại nhị phân (Rời bỏ/Ở lại) dựa trên bộ dữ liệu có cấu trúc từ Kaggle (gồm nhân khẩu học, tài chính và hành vi). Đề tài giới hạn phạm vi ở việc áp dụng các thuật toán Học máy (Machine Learning) truyền thống, không mở rộng sang các mô hình Học sâu (Deep Learning) hay xử lý dữ liệu phi cấu trúc.

2 Các nghiên cứu liên quan

2.1 Tổng quan các nghiên cứu về dự đoán Customer Churn trong lĩnh vực Ngân hàng

Dự đoán Customer Churn là bài toán trọng tâm trong ngành ngân hàng do tác động trực tiếp đến lợi nhuận [13]. Trước đây, các mô hình thống kê như Logistic Regression được ưa chuộng nhờ khả năng diễn giải tốt, nhưng lại hạn chế khi xử lý dữ liệu phức tạp [13]. Gần đây, xu hướng áp dụng Học máy (Machine Learning) đã chứng minh hiệu quả vượt trội trong việc nâng cao độ chính xác dự báo, đồng thời giúp xác định rõ các yếu tố tác động chính như tuổi tác, tài chính và hành vi sử dụng sản phẩm [8].

2.2 Các phương pháp và thuật toán phân lớp trong bài toán Customer Churn

Nhiều thuật toán phân lớp đã được áp dụng cho bài toán dự đoán Customer Churn, trong đó có thể kể đến: Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, XGBoost, LightGBM.

Ngoài ra, một số nghiên cứu cũng áp dụng mạng nơ-ron nhân tạo (Neural Networks) để khai thác các mối quan hệ phi tuyến phức tạp, tuy nhiên các mô hình này thường khó diễn giải và ít được ưu tiên trong bối cảnh ngân hàng [8].

2.3 Điểm mới và hướng tiếp cận của nhóm nghiên cứu

Khắc phục hạn chế của các nghiên cứu trước thường chỉ dựa trên độ chính xác đơn thuần (Accuracy), đề tài tiếp cận theo hướng so sánh toàn diện hiệu suất của nhiều mô hình học máy trên cùng bộ dữ liệu. Điểm mới nằm ở việc ưu tiên tối ưu hóa các chỉ số thực tiễn cho dữ liệu mất cân bằng (Recall, ROC-AUC) để tối đa hóa khả năng phát hiện rủi ro. Đồng thời, nghiên cứu đi sâu phân tích mức độ quan trọng đặc trưng (Feature Importance) nhằm chuyển hóa kết quả kỹ thuật thành các chiến lược quản trị khách hàng cụ thể.

3 Phương pháp

Nhóm nghiên cứu sử dụng 6 thuật toán phân lớp phổ biến:

3.1 Logistic Regression

Cơ sở lý thuyết. Hồi quy Logistic (Logistic Regression)[1,7] thực chất là một thuật toán phân loại (Classification) dựa trên xác suất. Mô hình này giải quyết bài toán dự báo biến phụ thuộc nhị phân Y ($Y \in \{0, 1\}$) dựa trên tập hợp các biến độc lập X .

Hàm giả thuyết (Hypothesis Function): Thay vì khớp một đường thẳng như hồi quy tuyến tính (dễ dẫn đến giá trị dự đoán nằm ngoài khoảng $[0,1]$), Hồi quy Logistic sử dụng hàm kích hoạt Sigmoid (Logistic Function) để nén đầu ra vào khoảng xác suất hợp lệ. Hàm số được định nghĩa:

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

Trong đó z là tổ hợp tuyến tính của các đặc trưng đầu vào và trọng số mô hình:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (2)$$

Ước lượng tham số (Parameter Estimation): Để tìm ra bộ tham số β tối ưu, thuật toán không dùng phương pháp Bình phương tối thiểu (OLS) như trong Linear Regression mà sử dụng phương pháp Hợp lý cực đại (Maximum Likelihood Estimation - MLE). Mục tiêu là tối đa hóa khả năng xảy ra của dữ liệu quan sát được, tương đương với việc tối thiểu hóa hàm mất mát Log-Loss (Cross-Entropy):

$$J(\beta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\beta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\beta}(x^{(i)})) \right] \quad (3)$$

Kiến trúc mô hình. Quy trình xây dựng mô hình gồm 3 giai đoạn chính:

- Tiền xử lý và Kiểm định giả định (Assumption Checking): Trước khi huấn luyện, dữ liệu được chuẩn hóa về dạng số thực (*float*) để đảm bảo tính ổn định trong tính toán ma trận. Nghiên cứu thực hiện kiểm định Đa cộng tuyến (Multicollinearity Check) bằng chỉ số VIF (Variance Inflation Factor).

Công thức: $VIF_i = \frac{1}{1-R_i^2}$, với R_i^2 là hệ số xác định khi hồi quy biến theo các biến còn lại. Tiêu chuẩn: Các biến có $VIF > 5$ sẽ bị loại bỏ. Điều này ngăn chặn hiện tượng phương sai của hệ số hồi quy bị thổi phồng, giúp mô hình ổn định và dễ giải thích hơn.[6]

- Tối ưu hóa Siêu tham số (Hyperparameter Tuning): Nghiên cứu sử dụng kỹ thuật tìm kiếm lưới Grid Search kết hợp với Kiểm định chéo (5-fold Cross-Validation). Xử lý mất cân bằng lớp (Class Imbalance): Thay vì sử dụng kỹ thuật lấy mẫu (Sampling), nghiên cứu áp dụng phương pháp Cost-sensitive Learning thông qua tham số `class_weight='balanced'`. Mô hình sẽ gán trọng số phạt w_j cho lớp j tỷ lệ nghịch với tần suất xuất hiện của nó:

$$w_j = \frac{n_{samples}}{n_{classes} \times n_{samples_j}} \quad (4)$$

- Hậu xử lý ngưỡng quyết định (Threshold Tuning): Kết quả đầu ra của Logistic Regression là xác suất (P). Việc chuyển đổi từ xác suất sang nhãn (0 hoặc 1) phụ thuộc vào ngưỡng cắt τ : Thay vì sử dụng ngưỡng mặc định $\tau = 0.5$, nghiên cứu tối ưu hóa ngưỡng dựa trên đường cong Precision-Recall. Ngưỡng tối ưu τ_{best} được chọn tại điểm cực đại hóa chỉ số F1-Score (trung bình điều hòa của Precision và Recall), nhằm cân bằng giữa việc không bỏ sót khách hàng rời bỏ và hạn chế báo động giả.

Thông số thực nghiệm. Các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:.

Bảng 1. Thông số thực nghiệm mô hình Logistic Regression.

Tham số	Giá trị	Lý giải
Solver	liblinear	Tối ưu cho dữ liệu nhỏ/trung bình, hội tụ nhanh.[12]
Regularization(C)	[0.01, 0.1, 1, 10, 100]	Tham số C là nghịch đảo của sức mạnh điều chuẩn: C nhỏ (0.01, 0.1) - Tăng cường Regularization, giúp giảm phương sai, tránh Overfitting nhưng có thể gây Underfitting. C lớn (10, 100) - Giảm Regularization, cho phép mô hình khớp chặt hơn với dữ liệu huấn luyện.
Penalty	L2	Sử dụng chuẩn L2 (Ridge Regression) để phạt các hệ số trọng số quá lớn, giữ cho mô hình mượt mà.
Scoring Metric	roc_auc	Sử dụng Diện tích dưới đường cong ROC làm tiêu chí chọn mô hình tốt nhất trong GridSearch, vì chỉ số này đánh giá khả năng phân tách lớp tổng quát tốt hơn Accuracy trong bối cảnh dữ liệu mất cân bằng.
VIF Threshold	< 5.0	Ngưỡng an toàn để loại bỏ đa cộng tuyến.

3.2 KNN

Cơ sở lý thuyết. K-Nearest Neighbors (KNN)[4] là một thuật toán học máy giám sát thuộc nhóm "học lười"(lazy learning) và dựa trên n-instance (instance-based learning). Không giống như các mô hình tham số (như Logistic Regression), KNN không học một hàm cố định từ dữ liệu huấn luyện mà thay vào đó, nó ghi nhớ toàn bộ dữ liệu. Khi có một điểm dữ liệu mới cần dự đoán, thuật toán sẽ tìm kiếm K điểm dữ liệu gần nhất trong không gian đặc trưng và đưa ra quyết định dựa trên nguyên tắc đa số (majority voting) hoặc tính trọng số khoảng cách.

Để xác định độ "gần" giữa các điểm dữ liệu, mô hình sử dụng các phép đo khoảng cách, phổ biến nhất là khoảng cách Euclidean và Manhattan.

Khoảng cách Euclidean ($p = 2$):

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (5)$$

Khoảng cách Manhattan ($p = 1$):

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (6)$$

Kiến trúc mô hình. Quy trình xây dựng mô hình gồm 3 giai đoạn chính:

- Tìm kiếm siêu tham số kép (Dual Hyperparameter Search Strategy):
Nghiên cứu kết hợp hai phương pháp để tìm giá trị K tối ưu:
Phương pháp Elbow (Khủy tay): Trực quan hóa mối quan hệ giữa giá trị K (từ 1 đến 30) và tỷ lệ lỗi trên biểu đồ để xác định vùng giá trị khả thi, tránh Underfitting (K quá lớn) hoặc Overfitting (K quá nhỏ).
Grid Search Cross-Validation: Sau khi quan sát biểu đồ Elbow, thuật toán GridSearch được áp dụng để thử nghiệm tổ hợp các tham số chính xác (số láng giềng, trọng số, loại khoảng cách) dựa trên điểm số F1-Score.
- Tối ưu ngưỡng quyết định (Threshold Tuning): Nghiên cứu trích xuất xác suất dự báo (P) và tối ưu hóa ngưỡng cắt τ dựa trên đường cong Precision-Recall. Mục tiêu: Tìm ngưỡng τ_{best} để tối đa hóa F1-Score, giúp cân bằng giữa độ chính xác và độ phủ trong việc phát hiện khách hàng rời bỏ.
- Giải thích mô hình bằng Permutation Importance:
Nguyên lý: Hoán đổi ngẫu nhiên giá trị của một đặc trưng trong tập kiểm tra và đo lường mức độ sụt giảm của F1-Score. Nếu F1-Score giảm mạnh sau khi hoán đổi, đặc trưng đó được coi là quan trọng đối với khả năng dự báo của mô hình.[2,11]

Thông số thực nghiệm. Các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:

Bảng 2. Thông số thực nghiệm mô hình KNN.

Tham số	Giá trị	Lý giải
K	[3, 5, 7, 9, 11, 15, 21]	Số lượng láng giềng tham gia bỏ phiếu. Chỉ chọn số lẻ để tránh tình trạng hòa phiếu (tie-breaking) trong bài toán phân loại nhị phân.
weights	['uniform', 'distance']	Cách tính trọng số phiếu bầu: Uniform: Mọi láng giềng có quyền lực ngang nhau. Distance: Láng giềng gần hơn có trọng số lớn hơn (nghịch đảo khoảng cách).
Metric (p)	[1, 2]	$p = 1$: Manhattan Distance (Tốt cho không gian nhiều chiều). $p = 2$: Euclidean Distance (Khoảng cách hình học thông thường).
Scoring Metric	f1	Sử dụng F1-Score làm tiêu chí tối ưu hóa trong GridSearch thay vì Accuracy, do đặc thù bài toán phát hiện rời bỏ (Churn) cần sự cân bằng giữa Precision và Recall.
Permutation Repeats	10	Số lần lặp lại việc xáo trộn dữ liệu khi tính toán độ quan trọng đặc trưng để đảm bảo kết quả ổn định.

3.3 Decision Tree

Cơ sở lý thuyết. Decision Tree là mô hình phân cấp dựa trên các quy tắc điều kiện (If-Then) để phân chia dữ liệu thành các tập con có độ thuần khiết cao nhất. Quá trình phân chia lặp lại từ nút gốc qua các nút quyết định dựa trên thuộc tính đầu vào và kết thúc tại nút lá, nơi đưa ra kết quả dự báo.

Cơ sở toán học: Các tiêu chuẩn phân chia (Splitting Criteria). Để xây dựng cây tối ưu, tại mỗi nút, thuật toán cần đánh giá độ thuần khiết (purity) của dữ liệu. Nghiên cứu xem xét theo thước đo độ hỗn loạn thông tin được hỗ trợ bởi thư viện Scikit-Learn:

- Độ hỗn loạn thông tin (Entropy): Entropy đo lường mức độ không chắc chắn (uncertainty) của phân phối nhãn tại một nút:

$$\text{Entropy}(t) = - \sum_{i=1}^c p(i|t) \log_2 p(i|t) \quad (7)$$

Giá trị Entropy càng nhỏ thì mức độ thuần khiết của nút càng cao, và đạt giá trị bằng 0 khi tất cả các mẫu tại nút đều thuộc cùng một lớp, và đạt giá trị lớn nhất khi phân phối các lớp là đồng đều.

- Phương trình chọn đặc trưng tối ưu (Feature Selection Criteria) Tại mỗi bước phân chia, thuật toán sẽ duyệt qua tất cả các thuộc tính A. Thuật toán tìm cách tối đa hóa độ lợi thông tin (Information Gain), tương đương với việc giảm entropy nhiều nhất sau khi phân chia:

$$A^* = \arg \max_A \left(\text{Entropy}(D) - \sum_{j=1}^k \frac{|D_j|}{|D|} \text{Entropy}(D_j) \right) \quad (8)$$

Thông số thực nghiệm. Các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:

Bảng 3. Thông số thực nghiệm mô hình Decision Tree.

Tham số	Giá trị	Lý giải
Criterion	entropy	Thuật toán ưu tiên tối đa hóa độ lợi thông tin (Information Gain) thay vì sử dụng Gini.
Max_depth	10	Độ sâu cây được giới hạn ở mức 10, giúp mô hình nắm bắt đủ các mẫu hành vi phức tạp nhưng ngăn chặn được hiện tượng học vẹt (overfitting).
Min_samples_split	20	Yêu cầu một nút phải chứa ít nhất 20 mẫu dữ liệu mới được phép phân chia tiếp, giúp loại bỏ các nhánh nhiều chi tiết.
Random_state	42	Cố định hạt giống ngẫu nhiên để đảm bảo kết quả thực nghiệm có tính nhất quán và tái lập được.

Quy trình huấn luyện.

- Xác định không gian tham số cần tìm kiếm và lựa chọn chỉ số F1-Score làm thước đo đánh giá chính để xử lý vấn đề mất cân bằng dữ liệu.
- Tối ưu hóa tham số bằng GridSearchCV: Khởi tạo mô hình Decision Tree và thực hiện tìm kiếm lưới kết hợp kiểm định chéo (5-fold Cross-Validation) để xác định bộ tham số tốt nhất
- Huấn luyện mô hình theo cơ chế phân hoạch đệ quy. Quá trình xây dựng cây được thực hiện lặp lại tại mỗi nút phân chia:
Tính toán Độ lợi thông tin cho tất cả thuộc tính.
Chọn thuộc tính A^* tối ưu giúp cực đại hóa luồng thông tin thu được:

$$A^* = \arg \max_A \left(Entropy(D) - \sum_{j=1}^k \frac{|D_j|}{|D|} Entropy(D_j) \right) \quad (9)$$

Phân chia dữ liệu và lặp lại cho đến khi đạt điều kiện dừng (độ sâu tối đa hoặc số mẫu tối thiểu).

- Dự đoán và Đánh giá: Sử dụng mô hình tối ưu để dự báo trên tập kiểm thử (Test Set) và kiểm chứng hiệu quả thông qua Ma trận nhầm lẫn (Confusion Matrix) cùng các chỉ số Accuracy, Precision, Recall, F1-Score.

3.4 Random Forest

Cơ sở lý thuyết. Random Forest là một phương pháp học máy tổ hợp (Ensemble Learning) dựa trên kỹ thuật Bagging (Bootstrap Aggregating). Thay vì phụ thuộc vào một cây quyết định duy nhất, Random Forest xây dựng một tập hợp gồm nhiều cây quyết định hoạt động song song và độc lập để đưa ra kết quả cuối cùng.[2]

Kiến trúc mô hình. Kiến trúc mô hình dựa trên hai cơ chế ngẫu nhiên hóa chính giúp giảm phương sai (variance) và hạn chế hiện tượng quá khớp (overfitting):

- Lấy mẫu dòng (Bootstrap Sampling): Mỗi cây con được huấn luyện trên một tập dữ liệu con được lấy mẫu ngẫu nhiên có hoàn lại từ tập dữ liệu gốc.
- Lấy mẫu đặc trưng (Feature Randomness): Tại mỗi nút phân chia của cây, thuật toán chỉ xem xét một tập hợp con ngẫu nhiên các đặc trưng thay vì toàn bộ đặc trưng.

Thông số thực nghiệm. Dựa trên kết quả tối ưu hóa (*rf_grid.best_params_*), các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:

Bảng 4. Thông số thực nghiệm mô hình Random Forest.

Tham số	Giá trị	Lý giải
n_estimators	100	Sử dụng 100 cây quyết định, đủ để đảm bảo độ ổn định của mô hình theo luật số lớn mà không gây tốn kém chi phí tính toán.
Max_depth	10	Giới hạn độ sâu ở mức 10 giúp mô hình tránh học các chi tiết nhiễu (noise) quá sâu, góp phần giảm Overfitting.
Min_samples_split	2	Yêu cầu mỗi nút lá phải có ít nhất 2 mẫu, giúp cây có tính tổng quát cao hơn so với việc để mặc định là 1.
class_weight	balanced	Tự động cân bằng trọng số giữa lớp Rời bỏ và Ở lại, giúp cải thiện khả năng nhận diện nhóm khách hàng Rời bỏ.

Quy trình huấn luyện.

- Khởi tạo, thiết lập chiến lược và Tối ưu hóa tham số (Hyperparameter Tuning): Thiết lập chiến lược xử lý mất cân bằng dữ liệu (*class_weight = 'balanced'*) và sử dụng GridSearchCV kết hợp kiểm định chéo để tìm ra bộ tham số tối ưu (*n_estimators, max_depth*) dựa trên chỉ số F1-Score.
- Huấn luyện tập hợp theo cơ chế Bagging (Bootstrap Aggregating):
Mô hình xây dựng song song N cây quyết định độc lập.
Mỗi cây được huấn luyện trên một tập dữ liệu con được lấy mẫu ngẫu nhiên có hoàn lại (Bootstrap sample) từ tập dữ liệu gốc.
Tại mỗi nút phân chia, thuật toán chỉ xem xét một tập hợp ngẫu nhiên các đặc trưng (Feature Randomness) để giảm sự tương quan giữa các cây.
- Tổng hợp kết quả và Kiểm định độ ổn định:

Kết quả dự báo cuối cùng được xác định thông qua cơ chế Bầu chọn đa số (Majority Voting) từ tất cả các cây thành phần.

Mô hình tối ưu sau cùng được kiểm tra mức độ quá khớp (Overfitting) bằng cách so sánh độ chính xác trên tập huấn luyện và tập kiểm thử trước khi đưa vào đánh giá chi tiết.

3.5 XGBoost

Cơ sở lý thuyết. XGBoost (eXtreme Gradient Boosting)[5] là thuật toán thuộc nhóm Gradient Boosting Machines (GBM), sử dụng tập hợp nhiều cây quyết định được xây dựng tuần tự nhằm giảm dần sai số của mô hình. Ở mỗi vòng lặp boosting, một cây mới được sinh ra để mô hình hóa phần sai số còn lại của các cây trước đó. Dự đoán cuối cùng là tổng các giá trị của K cây:

$$\hat{y}_i = \sum_{t=1}^K f_t(x_i), \quad f_t \in \mathcal{F} \quad (10)$$

Trong bài toán phân loại nhị phân (dự đoán rời bỏ khách hàng), XGBoost sử dụng hàm mất mát dạng Log-Loss để tính toán sai số và cập nhật mô hình.

Nguyên lý tối ưu hóa. Quá trình huấn luyện XGBoost tối ưu hàm mục tiêu tổng quát:

$$Obj = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{t=1}^K \Omega(f_t) \quad (11)$$

Hàm mất mát L : Đối với phân loại nhị phân, thường sử dụng Log-Loss.

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (12)$$

Hàm chính quy hóa :Giúp giảm overfitting và kiểm soát độ phức tạp của cây.

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (13)$$

XGBoost sử dụng đạo hàm bậc nhất và bậc hai (second-order gradient) để tối ưu hoá, giúp tăng tốc độ hội tụ và cải thiện tính ổn định.

Thông số thực nghiệm. Các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:

Bảng 5. Thông số thực nghiệm mô hình XGBoost.

Tham số	Giá trị	Lý giải
n_estimators	[100, 200, 300]	Số lượng cây trong ensemble.
Max_depth	[3, 5, 7]	Độ sâu tối đa của mỗi cây quyết định.
learning_rate	[0.05, 0.1]	Tốc độ học, kiểm soát đóng góp của mỗi cây.
random_state	42	Đảm bảo khả năng tái lập.
eval_metric	logloss	Hàm đánh giá trong huấn luyện.

Những tham số trên được lựa chọn nhằm cân bằng giữa khả năng học của mô hình và nguy cơ overfitting.[14]

Xử lý mất cân bằng dữ liệu: Dữ liệu phân loại rời bỏ khách hàng thường mất cân bằng, trong đó lớp rời bỏ chiếm tỷ trọng rất nhỏ. Do đó, mô hình áp dụng tham số: $scale_pos_weight = \frac{\text{số lượng mẫu lớp 0}}{\text{số lượng mẫu lớp 1}}$. Tham số này tăng trọng số cho lớp thiểu số, giúp mô hình nhạy hơn với các trường hợp khách hàng rời bỏ, từ đó cải thiện F1-score và khả năng nhận diện churn.

Trong mô hình hiện tại, hệ số cân bằng được tính tự động: $scale_pos_weight = ratio_value$

Quy trình huấn luyện.

- Tính toán tỷ lệ mất cân bằng và thiết lập tham số $scale_pos_weight$.
- Khởi tạo mô hình XGBoost và tối ưu tham số bằng GridSearchCV.
- Huấn luyện mô hình XGBoost theo cơ chế boosting. Vòng lặp boosting ($t = 1$ đến K):
 Tính gradient bậc nhất và bậc hai của hàm mất mát.
 Xây dựng cây quyết định f_t tối ưu hàm mục tiêu.
 Cập nhật mô hình bằng cách cộng thêm đóng góp của cây mới vào dự đoán hiện tại:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \quad (14)$$
- Dự đoán cuối cùng: Mô hình tối ưu được sử dụng để dự đoán xác suất và nhãn trên tập kiểm tra. Các chỉ số như Accuracy, F1-score, Confusion Matrix và ROC-AUC được tính toán để đánh giá hiệu suất mô hình.

3.6 LightGBM

Cơ sở lý thuyết. LightGBM (Light Gradient Boosting Machine)[10] là một thuật toán thuộc họ Gradient Boosting, sử dụng cơ chế xây dựng cây theo hướng leaf-wise giúp tìm ra các điểm tách tối ưu và giảm hàm mất mát nhanh hơn so với các phương pháp depth-wise truyền thống.. Trong bài toán phân loại nhị phân dự đoán Churn (0 – giữ lại, 1 – rời bỏ), LightGBM tạo ra một mô hình tổ

hợp dựa trên K cây quyết định, trong đó dự đoán cuối cùng là tổng đóng góp của từng cây:

$$\hat{y}_i = \sum_{t=1}^K f_t(x_i), \quad f_t \in \mathcal{F} \quad (15)$$

Mỗi cây trong LightGBM học cách tách lá có giảm loss lớn nhất, giúp giảm tối đa hàm mất mát tổng thể nhanh hơn phương pháp depth-wise truyền thống.

Nguyên lý tối ưu hóa. Mục tiêu huấn luyện của LightGBM là cực tiểu hóa hàm mục tiêu tổng quát:

$$Obj = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{t=1}^K \Omega(f_t) \quad (16)$$

Hàm mất mát L : Đối với phân loại nhị phân, thường sử dụng Log-Loss.

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (17)$$

Hàm chính quy hóa :Giúp giảm overfitting và kiểm soát độ phức tạp của cây.

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (18)$$

Hàm chính quy hóa giúp giảm overfitting và kiểm soát độ phức tạp của cây.

Thông số thực nghiệm. Các tham số tối ưu được lựa chọn cho mô hình được trình bày trong Bảng sau:

Bảng 6. Thông số thực nghiệm mô hình LightGBM.

Tham số	Giá trị	Lý giải
n_estimators	[100, 200, 300]	Số lượng cây trong ensemble.
Max_depth	[-1, 10, 20]	Không giới hạn chiều sâu cây.
learning_rate	[0.05, 0.1]	Tốc độ học, kiểm soát đóng góp của mỗi cây.
random_state	42	Đảm bảo khả năng tái lập.
num_leaves	[31, 50]	Số lá tối đa cho mỗi cây.
verbose	-1	Tắt thông báo trong quá trình huấn luyện.

Xử lý mất cân bằng dữ liệu: Trong quá trình khởi tạo mô hình, nhóm sử dụng tham số `class_weight = 'balanced'`. Cơ chế này giúp LightGBM tự động tính toán và gán trọng số cao hơn cho lớp thiểu số (khách hàng rời bỏ), nhằm giảm sai lệch do dữ liệu mất cân bằng và cải thiện Recall của mô hình.

Quy trình huấn luyện.

- Xử lý mất cân bằng dữ liệu bằng *class_weight*.
- Khởi tạo mô hình LightGBM và tối ưu tham số bằng GridSearchCV.
- Huấn luyện mô hình LightGBM theo cơ chế boosting. Vòng lặp boosting ($t = 1$ đến K):

Tính gradient bậc nhất và bậc hai của hàm mất mát.

Xây dựng cây quyết định tối ưu dựa trên cơ chế leaf-wise growth, chọn nhánh có mức giảm loss lớn nhất.

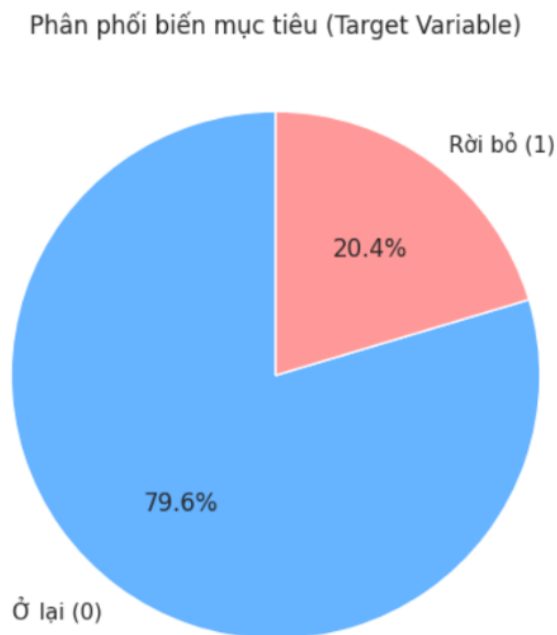
Cập nhật mô hình bằng cách cộng thêm đóng góp của cây mới vào dự đoán hiện tại:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \quad (19)$$

- Dự đoán và đánh giá bằng các chỉ số phân loại như Accuracy, F1-score, ROC-AUC.

4 Thực nghiệm**4.1 Bộ dữ liệu****Mô tả, khám phá bộ dữ liệu ban đầu.**

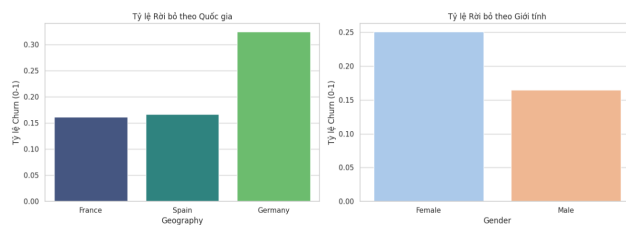
- Bộ dữ liệu: "Bank Customer Churn Prediction Dataset" (Dữ liệu Dự đoán Khách hàng Rời bỏ Ngân hàng).
- Nguồn thu thập: Kaggle
Kích thước dữ liệu: 10.000 dòng (quan sát) và 14 cột (thuộc tính).
Mục tiêu phân tích: Dựa trên các đặc điểm của khách hàng để dự báo khả năng họ sẽ rời bỏ ngân hàng (Churn) hay tiếp tục sử dụng dịch vụ.
- Thông tin các thuộc tính: Sau khi loại bỏ các thuộc tính định danh không có giá trị dự báo (gồm *RowNumber*, *CustomerId*, *Surname*), các biến quan trọng được sử dụng trong nghiên cứu bao gồm: *CreditScore*, *Geography*, *Gender*, *Age*, *Tenure*, *Balance*, *NumOfProducts*, *HasCrCard*, *IsActiveMember*, *EstimatedSalary*, *Exited*
- Thống kê mô tả: Bộ dữ liệu thường cho thấy một sự mất cân bằng về lớp (Class Imbalance) đối với biến mục tiêu:
Tỷ lệ Churn ($Exited = 1$): Khoảng 20,4% khách hàng rời bỏ ngân hàng.
Tỷ lệ Retained ($Exited = 0$): Khoảng 79,6% khách hàng được giữ lại.



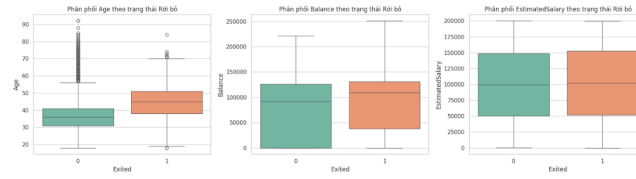
Hình 1. Thống kê mô tả.

– Trực quan hóa dữ liệu (Data Visualization):

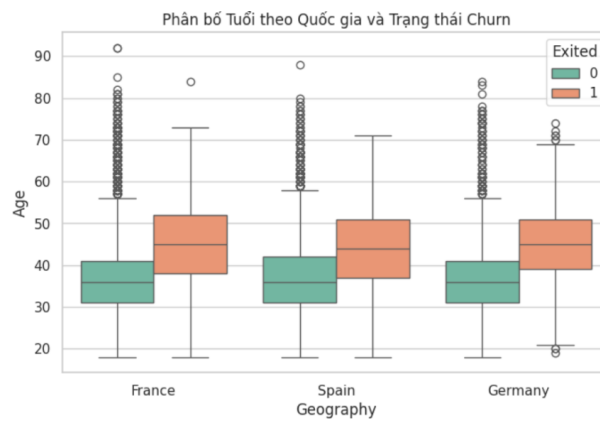
Để hiểu rõ hơn về hồ sơ khách hàng rời bỏ, nhóm thực hiện trực quan hóa các mối quan hệ giữa biến mục tiêu và các biến đặc trưng theo các yếu tố: Nhân khẩu học (Quốc gia - Giới tính), Biến định lượng (Độ tuổi, Số dư) và Đa biến (Tuổi - Quốc gia - Rời bỏ)



Hình 2. Tỷ lệ rời bỏ theo Quốc gia (trái) và Giới tính (phải).



Hình 3. Phân phối độ tuổi, số dư và lương theo trạng thái rời bỏ.



Hình 4. Mối quan hệ giữa Tuổi và Rời bỏ trên từng Quốc gia.

	RowNumber	CustomerId	Creditscore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
count	10000.000000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.500000	1.560904e+07	650.520600	38.921800	5.012900	76465.880288	1.530200	0.700500	0.515100	100090.236981	0.202790
std	2886.895668	7.113813e+04	96.653299	10.487806	2.892174	62397.405202	0.581054	0.45584	0.499797	57510.402818	0.402789
min	1.000000	1.558570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.000000	0.000000	11.580000	0.000000
25%	2500.750000	1.562833e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.000000	0.000000	51002.110000	0.000000
50%	5000.500000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.000000	1.000000	100193.915000	0.000000
75%	7500.250000	1.575323e+07	718.000000	44.000000	7.000000	127844.240000	2.000000	1.000000	1.000000	149388.247500	0.000000
max	10000.000000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.000000	1.000000	198992.480000	1.000000

Hình 5. Bảng thống kê mô tả dữ liệu.

Tiền xử lý, phân chia bộ dữ liệu. Làm sạch và chọn lọc đặc trưng: Dữ liệu được xác nhận không chứa giá trị thiếu. Các biến định danh không mang ý nghĩa dự báo (*RowNumber*, *CustomerId*, *Surname*) được loại bỏ để giảm nhiễu.

Mã hóa biến phân loại (Encoding): Áp dụng Label Encoding cho biến *Gender* và One-Hot Encoding cho biến *Geography* (có loại bỏ biến tham chiếu để tránh hiện tượng đa cộng tuyến).

Chuẩn hóa dữ liệu (Scaling): Các biến số (*CreditScore, Age, Balance, EstimatedSalary...*) được chuẩn hóa theo phương pháp Z-score để đưa về cùng thang đo, giúp thuật toán hội tụ nhanh hơn.

Phân chia dữ liệu (Splitting): Bộ dữ liệu được chia thành tập huấn luyện và kiểm tra theo tỷ lệ 80/20. Kỹ thuật lấy mẫu phân tầng (Stratified Sampling) được áp dụng để đảm bảo tỷ lệ khách hàng rời bỏ ($\sim 20\%$) được giữ nguyên trong cả hai tập, đảm bảo tính đại diện khi đánh giá mô hình.

4.2 Tiêu chí đánh giá

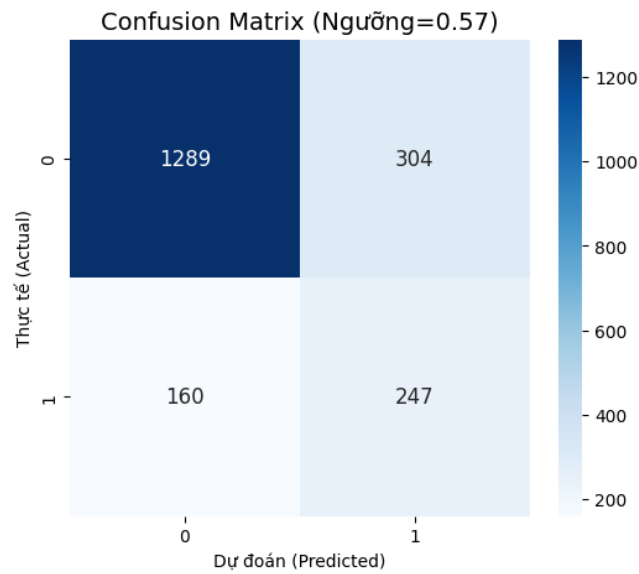
Bảng 7. Các độ đo được sử dụng.

Độ đo	Công thức	Ý nghĩa
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Tỷ lệ dự đoán đúng trên toàn bộ dữ liệu. Dễ hiểu nhưng chưa phản ánh tốt hiệu quả mô hình khi dữ liệu mất cân bằng.
Precision	$\frac{TP}{TP+FP}$	Trong số khách hàng được dự đoán là rời bỏ, có bao nhiêu người thực sự rời bỏ. Phù hợp khi chi phí chăm sóc sai đối tượng cao.
Recall	$\frac{TP}{TP+FN}$	Trong số khách hàng thực sự rời bỏ, mô hình phát hiện được bao nhiêu người. Thích hợp khi doanh nghiệp muốn giảm tối đa số khách hàng rời bỏ không được phát hiện.
F1-score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Trung bình điều hòa giữa Precision và Recall. Độ đo tổng hợp giúp cân bằng giữa việc dự đoán đúng và hạn chế bỏ sót khách hàng có nguy cơ rời bỏ.
ROC-AUC		Đánh giá khả năng phân biệt giữa hai nhóm khách hàng (rời bỏ và ở lại). Giá trị AUC càng gần 1 cho thấy mô hình phân loại càng tốt.

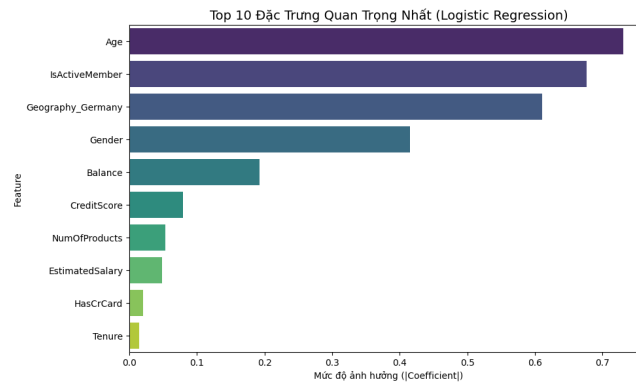
4.3 Kết quả thực nghiệm

Kết quả chi tiết từng mô hình.

Logistic Regression. Kết quả mô hình Logistic Regression:

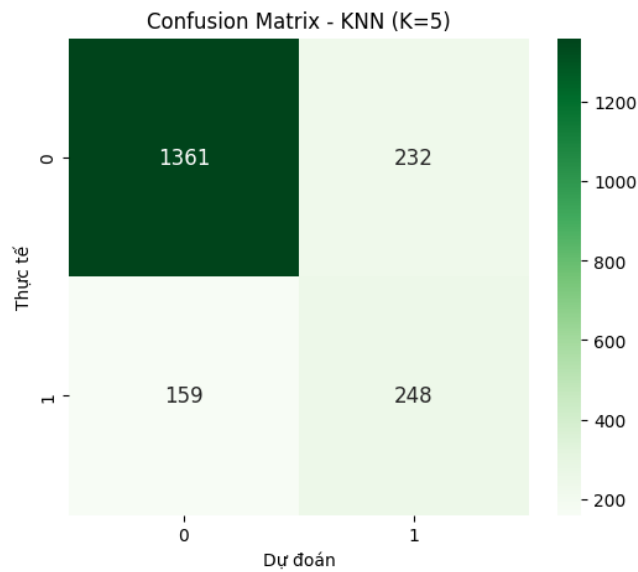


Hình 6. Ma trận nhầm lẫn của Logistic Regression.

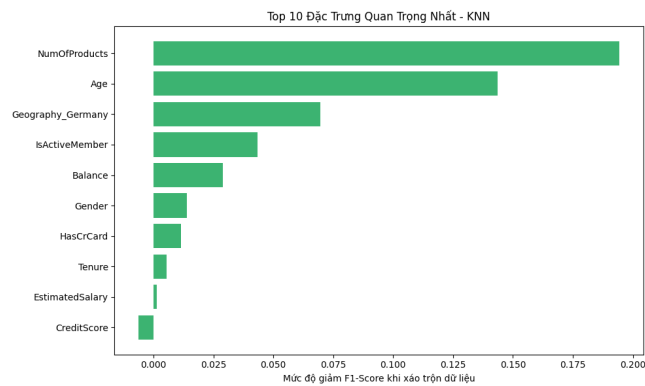


Hình 7. Tầm quan trọng đặc trưng của Logistic Regression.

KNN. Kết quả mô hình KNN:

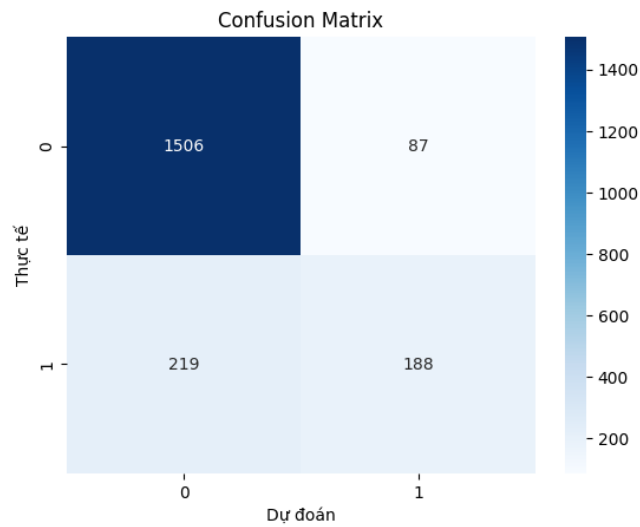


Hình 8. Ma trận nhầm lẫn của KNN.

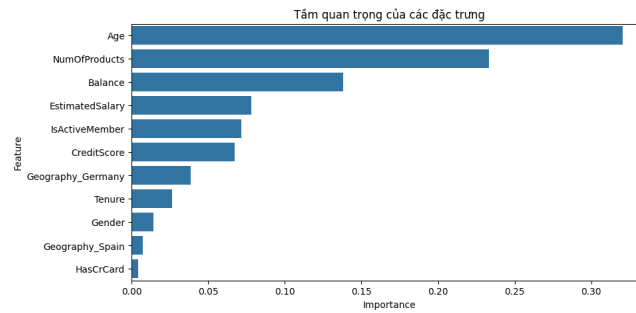


Hình 9. Tầm quan trọng đặc trưng của KNN.

Decision Tree. Kết quả mô hình Decision Tree:

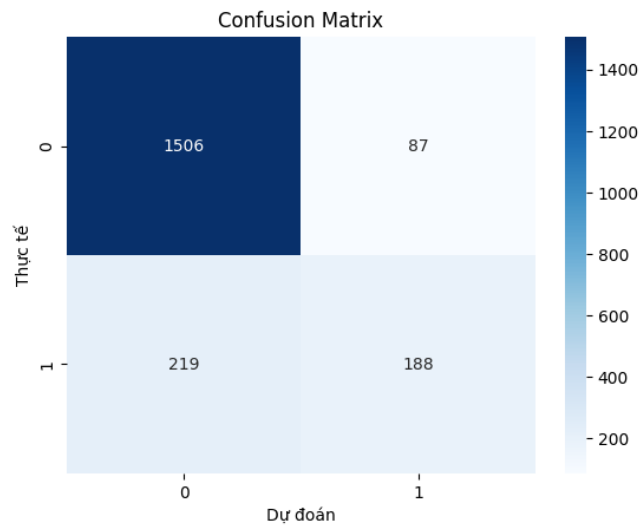


Hình 10. Ma trận nhầm lẫn của Decision Tree.

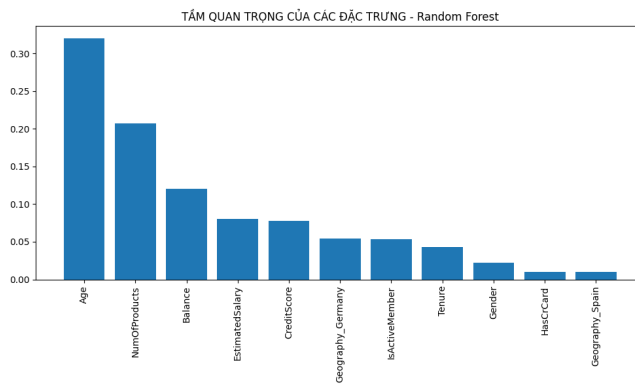


Hình 11. Tầm quan trọng đặc trưng của Decision Tree.

Random Forest. Kết quả mô hình Random Forest:

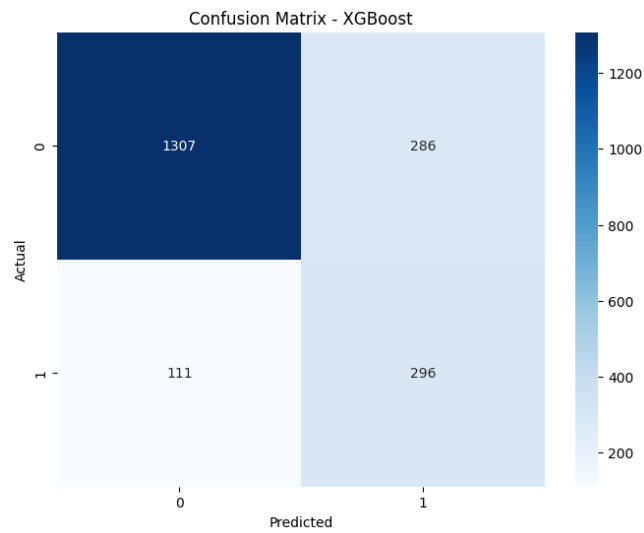


Hình 12. Ma trận nhầm lẫn của Random Forest.

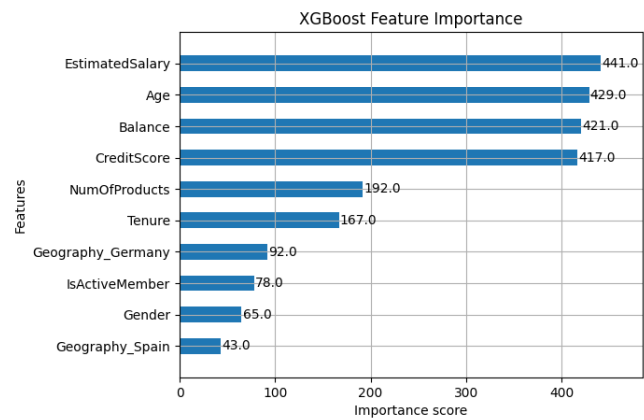


Hình 13. Tầm quan trọng đặc trưng của Random Forest.

XGBoost. Kết quả mô hình XGBoost:

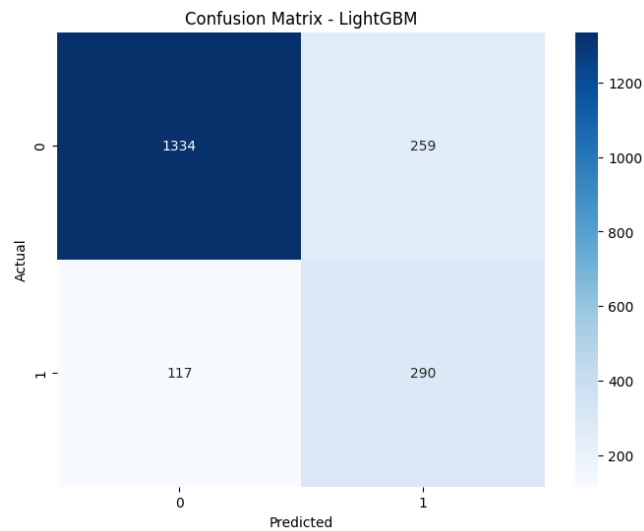


Hình 14. Ma trận nhầm lẫn của XGBoost.

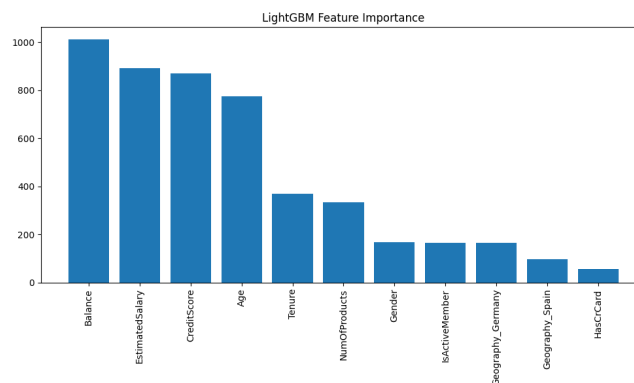


Hình 15. Tầm quan trọng đặc trưng của XGBoost.

LightGBM. Kết quả mô hình *LightGBM*:



Hình 16. Ma trận nhầm lẫn của LightGBM.



Hình 17. Tầm quan trọng đặc trưng của LightGBM.

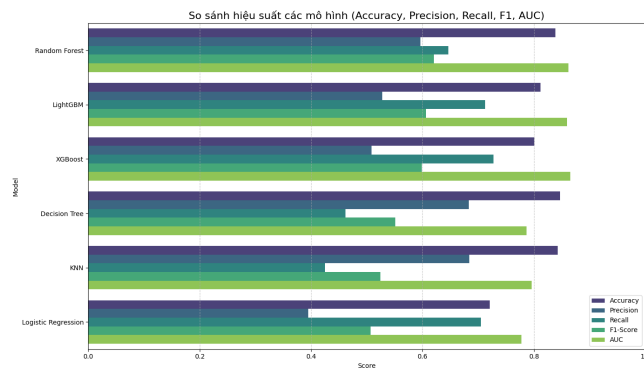
Đánh giá tổng quan 6 mô hình.

Sự phân hóa rõ rệt giữa hai nhóm mô hình:

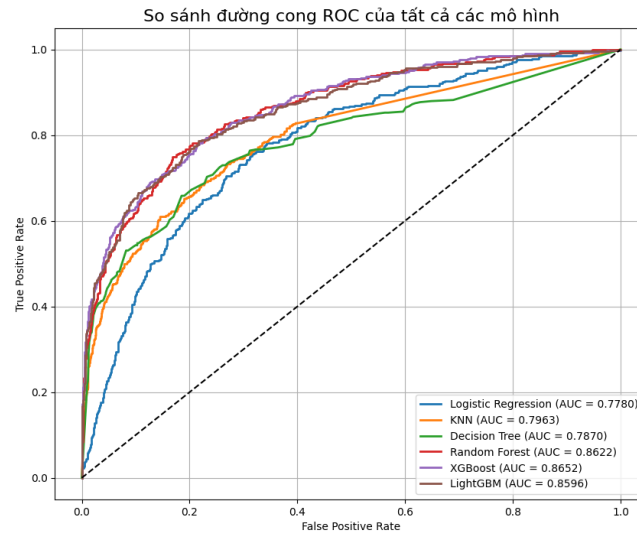
- Nhóm mô hình cơ bản (KNN, Decision Tree): Mắc phải "Nghịch lý độ chính xác" (Accuracy Paradox). Dù đạt Accuracy cao ($\sim 84.7\%$) nhưng Recall cực thấp (42-46%), thất bại trong việc nhận diện khách hàng rời bỏ do dữ liệu mất cân bằng.

	Model	Accuracy	Precision	Recall	F1-Score	AUC
3	Random Forest	0.839000	0.596372	0.646192	0.620283	0.862221
5	LightGBM	0.812000	0.528233	0.712531	0.606695	0.859590
4	XGBoost	0.801500	0.508591	0.727273	0.598584	0.865215
2	Decision Tree	0.847000	0.683636	0.461916	0.551320	0.787015
1	KNN	0.843000	0.683794	0.425061	0.524242	0.796299
0	Logistic Regression	0.720500	0.395317	0.705160	0.506620	0.777967

Hình 18. Bảng tổng hợp kết quả 6 mô hình.



Hình 19. So sánh hiệu suất 6 mô hình.



Hình 20. So sánh đường cong ROC của 6 mô hình.

- Nhóm mô hình nâng cao (Random Forest, XGBoost, LightGBM): Có Accuracy thấp hơn một chút (80% - 84%) nhưng Recall cao vượt trội (65% - 73%) và AUC rất cao (> 0.86).

Các chỉ số cốt lõi:

- Về Recall - Quan trọng nhất:
XGBoost (0.727): Tốt nhất - Bắt được gần 73% lượng khách hàng có ý định rời bỏ. LightGBM (0.713): Đứng sau XGBoost. Logistic Regression (0.705): Khá bất ngờ khi mô hình đơn giản này có Recall cao, nhưng đổi lại Precision quá thấp (0.395) - tức là "báo động giả" quá nhiều.
- Về độ tin cậy tổng thể (AUC Score) - đánh giá khả năng xếp hạng xác suất đúng sai của mô hình:
XGBoost (0.865) và Random Forest (0.862) là hai mô hình thống trị, cho thấy độ ổn định cao nhất. Decision Tree và Logistic Regression có AUC thấp nhất (< 0.79), cho thấy khả năng phân loại kém hơn.

Góc nhìn đa chiều từ Feature Importance: Trong khi Random Forest nhấn mạnh *Age*, LightGBM tập trung vào Tài chính (*Balance*, *EstimatedSalary*), thì XGBoost cân bằng tốt cả hai khía cạnh này. Điều này gợi ý chiến lược giữ chân cần tập trung vào nhóm khách hàng trung niên (40-60 tuổi) có biến động tài chính lớn.

5 Kết luận và Hướng phát triển

5.1 Kết luận

Sau quá trình thực nghiệm trên 06 thuật toán (Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost, LightGBM), nhóm quyết định lựa chọn XGBoost để triển khai thực tế.

- Hiệu suất vượt trội: XGBoost đạt Recall cao nhất (72.7%) – bắt đúng gần 73% khách hàng có ý định rời bỏ (cải thiện 26% so với mô hình cơ sở Decision Tree).
- Độ tin cậy cao: Chỉ số AUC đạt 0.865, chứng minh sự ổn định trong phân loại giữa hai lớp khách hàng, giải quyết tốt bài toán mất cân bằng dữ liệu.

5.2 Kiến nghị giải pháp

Nhóm Khách hàng Trung niên (Yếu tố *Age*): Thiết kế gói sản phẩm "Tích lũy hưu trí" với lãi suất ưu đãi. Triển khai luồng Hỗ trợ ưu tiên (Priority Support) tại quầy và tổng đài, đồng thời đơn giản hóa giao diện ứng dụng (Large UI mode) để phù hợp với thị hiếu nhóm tuổi này.

Nhóm Khách hàng Tài chính mạnh (Yếu tố *EstimatedSalary* & *Balance*): Xây dựng Hệ thống cảnh báo sớm (Early Warning System) trên CRM: tự động báo động cho nhân viên chăm sóc (RM) khi khách VIP giảm đột ngột số dư

hoặc ngưng giao dịch trong 1 tháng. Áp dụng chính sách Lãi suất bậc thang để khuyến khích duy trì số dư cao.

Tối ưu hóa hành trình sản phẩm (Yếu tố *NumOfProducts*): Với nhóm 1 sản phẩm: Đẩy mạnh Bán chéo (Cross-selling). Với nhóm 3-4 sản phẩm: Quy hoạch lại thành gói tài chính "All-in-one" để khách hàng dễ quản lý, giảm cảm giác bị phân tán dịch vụ.

Tài liệu

1. Bishop, C.M.: Pattern Recognition and Machine Learning. Springer, New York (2006)
2. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
3. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794. ACM (2016)
4. GeeksforGeeks: K-nearest neighbors (KNN) algorithm in machine learning (2026), <https://www.geeksforgeeks.org/k-nearest-neighbours/>, accessed: 04 Jan 2026
5. GeeksforGeeks: XGBoost - extreme gradient boosting (2026), <https://www.geeksforgeeks.org/xgboost/>, accessed: 04 Jan 2026
6. Hair, J.F., Black, W.C., Babin, B.J., Anderson, R.E.: Multivariate Data Analysis. Cengage Learning, Boston, 8th edn. (2019)
7. Hosmer, D.W., Lemeshow, S., Sturdivant, R.X.: Applied Logistic Regression. John Wiley & Sons, Hoboken, 3rd edn. (2013)
8. Idowu, M., Ojo, O.E., Adewale, O.O.: Customer churn prediction in the banking sector using supervised machine learning techniques. Journal of Big Data **7**(1), 1–21 (2020)
9. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 3149–3157 (2017)
10. LightGBM Developers: LightGBM documentation (2026), <https://lightgbm.readthedocs.io/en/stable/>, accessed: 04 Jan 2026
11. Scikit-learn Developers: Permutation feature importance (2026), https://scikit-learn.org/stable/modules/permutation_importance.html, accessed: 04 Jan 2026
12. Scikit-learn Developers: User guide: Logistic regression (2026), https://scikit-learn.org/stable/modules/linear_model.html#liblinear-solver, accessed: 04 Jan 2026
13. Verbeke, W., Dejaeger, K., Martens, D., Hur, J., Baesens, B.: New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. European Journal of Operational Research **218**(1), 211–229 (2012)
14. XGBoost Developers: XGBoost parameters (2026), <https://xgboost.readthedocs.io/en/stable/parameter.html>, accessed: 04 Jan 2026